Complexity Analysis

Big O, little o, theta, big Omega, and small omega notation

Definition

For real-valued nets f and g,

$\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M\,|g(x)|$ iff $f(x) \in O(g(x))$ and $|g(x)|$ is an asymptotic upper bound of $|f(x)|$.

$\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \geq M\,|g(x)|$ iff $f(x) \in \Omega(g(x))$ and $|g(x)|$ is an asymptotic lower bound of $|f(x)|$.

$\exists M_1, M_2 > 0, T \in \mathbb{R} : x \geq T \implies M_1\,|g(x)| \leq |f(x)| \leq M_2\,|g(x)|$ iff $f(x) \in \Theta(g(x))$.

$\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M\,|g(x)|$ iff $f(x) \in o(g(x))$.

$\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| > M\,|g(x)|$ iff $f(x) \in \omega(g(x))$.

$\in$ is also denoted as $=$ for them.

Limit

If $\exists k \in \mathbb{R} : x \geq k \implies g(x) \neq 0$,

$\limsup \left|\frac{f(x)}{g(x)}\right| < \infty$ iff $f(x) \in O(g(x))$.

Proof. $|f(x)| \leq M\,|g(x)|, \quad x \geq T; \left|\frac{f(x)}{g(x)}\right| \leq M, \quad x \geq T;$ $\sup \left|\frac{f(x)}{g(x)}\right| \leq M, \quad x \geq T; \limsup \left|\frac{f(x)}{g(x)}\right| \leq M < \infty;$ □

$\liminf \left|\frac{f(x)}{g(x)}\right| > 0$ iff $f(x) \in \Omega(g(x))$.

Proof. $|f(x)| \geq M\,|g(x)|, \quad x \geq T; \left|\frac{f(x)}{g(x)}\right| \geq M, \quad x \geq T;$ $\inf \left|\frac{f(x)}{g(x)}\right| \geq M, \quad x \geq T; \liminf \left|\frac{f(x)}{g(x)}\right| \geq M > 0;$ □

$\infty > \limsup \left|\frac{f(x)}{g(x)}\right| \geq \liminf \left|\frac{f(x)}{g(x)}\right| > 0$ iff $f(x) \in \Theta(g(x))$.

Proof. $|f(x)| \geq M_1\,|g(x)|, \quad x \geq T; \left|\frac{f(x)}{g(x)}\right| \geq M_1, \quad x \geq T;$ $\inf \left|\frac{f(x)}{g(x)}\right| \geq M_1, \quad x \geq T; \liminf \left|\frac{f(x)}{g(x)}\right| \geq M_1 > 0;$ $|f(x)| \leq M_2\,|g(x)|, \quad x \geq T; \left|\frac{f(x)}{g(x)}\right| \leq M_2, \quad x \geq T;$ $\sup \left|\frac{f(x)}{g(x)}\right| \leq M_2, \quad x \geq T; \limsup \left|\frac{f(x)}{g(x)}\right| \leq M_2 < \infty;$ □

$\lim_{x \to \infty} \left|\frac{f(x)}{g(x)}\right| = 0$ iff $f(x) \in o(g(x))$. $|f(x)| < M\,|g(x)|;$ $\left|\frac{f(x)}{g(x)}\right| < M; \lim_{x \to \infty} \left|\frac{f(x)}{g(x)}\right| = 0;$

$\lim_{x \to \infty} \left|\frac{f(x)}{g(x)}\right| = \infty$ iff $f(x) \in \omega(g(x))$. $|f(x)| > M\,|g(x)|;$ $\left|\frac{f(x)}{g(x)}\right| > M; \lim_{x \to \infty} \left|\frac{f(x)}{g(x)}\right| = \infty;$

Relation

For any fixed $g(x)$, $o(g(x))$, $\Theta(g(x))$, and $\omega(g(x))$ are mutually exclusive.

Proof. Prove by contradiction. If $f \in o(g(x)) \cap \Theta(g(x))$, $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M\,|g(x)|$; $\exists M_1, M_2 > 0, U \in \mathbb{R} : x \geq U \implies M_1\,|g(x)| \leq |f(x)| \leq M_2\,|g(x)|$; then set $M = M_1$, for $x \geq \max\{T, U\}$ $M_1\,|g(x)| \leq |f(x)| < M_1\,|g(x)|$.; which can't happen. If $f \in \omega(g(x)) \cap \Theta(g(x))$, $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| > M\,|g(x)|$; $\exists M_1, M_2 > 0, U \in \mathbb{R} : x \geq U \implies M_1\,|g(x)| \leq |f(x)| < M_2\,|g(x)|$; then set $M = M_2$, for $x \geq \max\{T, U\}$ $M_2\,|g(x)| \geq |f(x)| > M_2\,|g(x)|$.; which can't happen. If $f \in o(g(x)) \cap \omega(g(x))$, $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M\,|g(x)|$; $\forall M > 0 : \exists U \in \mathbb{R} : x \geq U \implies |f(x)| > M\,|g(x)|$; then set any fixed M, $M\,|g(x)| < |f(x)| < M\,|g(x)|$.; which can't happen. □

For any fixed $g(x)$, $o(g(x)) \subseteq O(g(x))$;

Proof. Peak any $M > 0$. □

For any fixed $g(x)$, $\omega(g(x)) \subseteq \Omega(g(x))$;

Proof. Peak any $M > 0$. □

For any fixed $g(x)$, $\Theta(g(x)) = O(g(x)) \cap \Omega(g(x))$;

Proof. Prove $\Theta(g(x)) \subseteq O(g(x)) \cap \Omega(g(x))$; $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M\,|g(x)|$; $\exists N > 0, U \in \mathbb{R} : x \geq U \implies |f(x)| \geq N\,|g(x)|$; then $\exists M, N > 0, \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies N\,|g(x)| \leq |f(x)| \leq M\,|g(x)|$;

Prove $\Theta(g(x)) \supseteq O(g(x)) \cap \Omega(g(x))$; $\exists M_1, M_2 > 0, T \in \mathbb{R} : x \geq T \implies M_1\,|g(x)| \leq |f(x)| \leq M_2\,|g(x)|$; then $\exists M_2 > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M_2\,|g(x)|$; $\exists M_1 > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \geq M_1\,|g(x)|$; □

For any fixed $g(x)$, $O(g(x)) \setminus (o(g(x)) \cup \Theta(g(x))) \neq \varnothing$;

Proof. Counterexample: $f(x) = \begin{cases} 1, & x \in \mathbb{Q} \\ 0, & x \notin \mathbb{Q} \end{cases}$; $g(x) = 1$; $f \in O(g(x))$: $\exists 1 > 0, 1 \in \mathbb{R} : x \geq 1 \implies |f(x)| \leq |g(x)|$; $f \notin o(g(x))$: $\forall 1 > 0, T \in \mathbb{R} : \exists x \geq T, x \in \mathbb{Q} : |f(x)| = |g(x)|$; $f \notin \Theta(g(x))$: $\forall M_1 > 0, x \in \mathbb{R}, x \notin \mathbb{Q} : M_1\,|g(x)| = M_1 > |f(x)| = 0$; □

For any fixed $g(x)$, $\Omega(g(x)) \setminus (\omega(g(x)) \cup \Theta(g(x))) \neq \varnothing$;

Proof. Counterexample: $f(x) = 1$; $g(x) = \begin{cases} 1, & x \in \mathbb{Q} \\ 0, & x \notin \mathbb{Q} \end{cases}$; $f \in \Omega(g(x))$: $\exists 1 > 0, 1 \in \mathbb{R} : x \geq 1 \implies |f(x)| \geq |g(x)|$; $f \notin \omega(g(x))$: $\forall 1 > 0, T \in \mathbb{R} : \exists x \geq T, x \in \mathbb{Q} : |f(x)| = |g(x)|$; $f \notin \Theta(g(x))$: $\forall M_2 > 0, x \in \mathbb{R}, x \notin \mathbb{Q} : M_2\,|g(x)| = 0 < |f(x)| = 1$; □

$f(x) \in O(g(x)) \iff g(x) \in \Omega(f(x))$;

Proof. $\implies$: $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M\,|g(x)|$; then $\exists \frac{1}{M} > 0, T \in \mathbb{R} : x \geq T \implies |g(x)| \geq \frac{1}{M}\,|g(x)|$; $\impliedby$: $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \geq M\,|g(x)|$; then $\exists \frac{1}{M} > 0, T \in \mathbb{R} : x \geq T \implies |g(x)| \leq \frac{1}{M}\,|g(x)|$; □

$f(x) \in o(g(x)) \iff g(x) \in \omega(f(x))$;

Proof. $\implies$: $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M\,|g(x)|$; then $\forall \frac{1}{M} > 0 : \exists T \in \mathbb{R} : x \geq T \implies |g(x)| > \frac{1}{M}\,|f(x)|$; $\impliedby$: $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| > M\,|g(x)|$; then $\forall \frac{1}{M} > 0 : \exists T \in \mathbb{R} : x \geq T \implies |g(x)| < \frac{1}{M}\,|f(x)|$; □

For any fixed $g(x)$, $\exists f(x) : f(x) \notin O(g(x)) \cup \Omega(g(x))$;

Proof. $f(x) = \begin{cases} 1, & x \in \mathbb{Q} \\ x, & x \notin \mathbb{Q} \end{cases}$; $g(x) = \begin{cases} x, & x \in \mathbb{Q} \\ 1, & x \notin \mathbb{Q} \end{cases}$; For $x \in \mathbb{R} \setminus \mathbb{Q}$, $\nexists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M\,|g(x)|$; For $x \in \mathbb{Q}$, $\nexists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \geq M\,|g(x)|$; □

Binary relation

$\Theta(g(x))$ is an equivalence class.

Proof. Transitive: $\exists M_1, M_2 > 0, T \in \mathbb{R} : x \geq T \implies M_1\,|g(x)| \leq |f(x)| \leq M_2\,|g(x)|$; $\exists N_1, N_2 > 0, U \in \mathbb{R} : x \geq U \implies N_1\,|h(x)| \leq |g(x)| \leq N_2\,|h(x)|$; then $\exists M_1 N_1, M_2 N_2 > 0, \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies M_1 N_1\,|h(x)| \leq |f(x)| \leq M_2 N_2\,|h(x)|$; Reflexive: $f(x) \in \Theta(f(x))$; $\exists 1, 1 > 0, a \in \mathbb{R} : x \in \mathbb{R} \implies 1\,|f(x)| \leq |f(x)| \leq 1\,|f(x)|$; Symmetric: $\exists M_1, M_2 > 0, T \in \mathbb{R} : x \geq T \implies M_1\,|g(x)| \leq |f(x)| \leq M_2\,|g(x)|$; then $\exists \frac{1}{M_2}, \frac{1}{M_1} > 0, T \in \mathbb{R} : x \geq T \implies \frac{1}{M_2}\,|f(x)| \leq |f(x)| \leq \frac{1}{M_1}\,|f(x)|$; □

$O(g(x))$ is partial-ordered set with Θ as $=$.

Proof. Transitive: $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M\,|g(x)|$; $\exists N > 0, U \in \mathbb{R} : x \geq U \implies |g(x)| \leq N\,|h(x)|$; then $\exists MN > 0, \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies |f(x)| \leq MN\,|h(x)|$; Reflexive: $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M\,|g(x)|$; then $\exists \frac{1}{M} > 0, T \in \mathbb{R} : x \geq T \implies |g(x)| \leq \frac{1}{M}\,|f(x)|$; Antisymmetric: $\exists M > 0, T \in \mathbb{R} : x \geq T \implies |f(x)| \leq M\,|g(x)|$; $\exists N > 0, U \in \mathbb{R} : x \geq U \implies |g(x)| \leq N\,|f(x)|$; then $\exists M, \frac{1}{N} > 0, \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies \frac{1}{N}\,|g(x)| \leq |f(x)| \leq M\,|g(x)|$; □

$\Omega(g(x))$ is partial-ordered set with Θ as $=$.

Proof. Similar to that of O. □

$o(g(x))$ is a strictly partially-ordered set.

Proof. Asymmetry: $f(x) \in o(g(x)) \iff g(x) \in \omega(f(x)) \implies g \notin o(f(x))$; Transitivity: $\forall M > 0 : \exists T \in \mathbb{R} : x \geq T \implies |f(x)| < M\,|g(x)|$; $\exists U \in \mathbb{R} : x \geq U \implies |g(x)| < |h(x)|$; then $\forall M > 0 : \exists \max\{T, U\} \in \mathbb{R} : x \geq \max\{T, U\} \implies |f(x)| < M\,|h(x)|$; □

$\omega(g(x))$ is a strictly partially-ordered set.

Proof. Similar to that of o. □

Master Theorem

Integral test (for convergence) or Maclaurin–Cauchy test

Suppose $f(x) : [a, \infty) \to \mathbb{R}$ for some $a \in \mathbb{N}$ is monotone and $\lim_{x \to \infty} f(x) = 0$. Then $\int_a^\infty f(x)\,\mathrm{d}x$ is convergent iff $\sum_{k=a}^{\infty} f(k)$ is convergent. And,

If $\int_a^\infty f(x)\,\mathrm{d}x$ is convergent and $f(x)$ is monotone decreasing, for any $n \in \mathbb{N}_{>a}$, $\int_n^\infty f(x)\,\mathrm{d}x \leq \sum_{k=n}^{\infty} f(k) \leq f(n) + \int_n^\infty f(x)\,\mathrm{d}x$.; called remainder estimate for integral test. Note that for $n \in \mathbb{N}_{>a}$, $f(n) + \int_n^\infty f(x)\,\mathrm{d}x \leq \int_{n-1}^\infty f(x)\,\mathrm{d}x$.

If $\int_a^\infty f(x)\,\mathrm{d}x$ is convergent and $f(x)$ is monotone increasing, for any $n \in \mathbb{N}_{\geq a}$, $\int_n^\infty f(x)\,\mathrm{d}x \geq \sum_{k=n}^{\infty} f(k) \geq f(n) + \int_n^\infty f(x)\,\mathrm{d}x$.; called remainder estimate for integral test. Note that for $n \in \mathbb{N}_{>a}$, $f(n) + \int_n^\infty f(x)\,\mathrm{d}x \geq \int_{n-1}^\infty f(x)\,\mathrm{d}x$.

Proof. Consider the case where $f(x)$ is monotone increasing. The case where $f(x)$ is monotone decreasing can be proven similarly. First consider the case where $\int_a^\infty f(x)\,\mathrm{d}x$ is convergent. This implies that $f(x)$ is non-positive. Thus, for any fixed $n \in \mathbb{N}_{\geq a}$, $\int_n^{n+1} f(x)\,\mathrm{d}x \geq f(n)$,; and for any $k \geq n + 1$, $\int_k^{k+1} f(x)\,\mathrm{d}x \geq f(k) \geq \int_{k-1}^k f(x)\,\mathrm{d}x$.; For any $N \in \mathbb{N}_{\geq n}$, summing over all $k \in \mathbb{N}_{\geq n}$, we obtain, $\int_n^{N+1} f(x)\,\mathrm{d}x \geq \sum_{k=n}^{N} f(k) \geq f(n) + \int_n^N f(x)\,\mathrm{d}x$.; Take limit $\lim_{N \to \infty}$, we obtain $\int_n^\infty f(x)\,\mathrm{d}x \geq \sum_{k=n}^{\infty} f(n) \geq f(n) + \int_n^\infty f(x)\,\mathrm{d}x$.; Second, consider the case where $\int_a^\infty f(x)\,\mathrm{d}x$ is divergent. Since $f(x)$ is monotone. Either $\int_a^\infty f(x)\,\mathrm{d}x = -\infty$ or $\int_a^\infty f(x)\,\mathrm{d}x = \infty$. For the former, the supremum of $f(x)$ is non-positive, and $\sum_{k=a}^{\infty} f(k) = -\infty$. For the latter, there exists $x_0 > a$ such that $f(x) > 0$ for all $x \geq x_0$, and thus $\sum_{k=a}^{\infty} f(k) = \infty$. □

p-test (for improper integral)

For any fixed $a > 0$, $\int_a^\infty \frac{1}{x^p}\,\mathrm{d}x$ converges iff $p > 1$, and if so, $\int_a^\infty \frac{1}{x^p}\,\mathrm{d}x = \frac{1}{(p-1)\,a^{p-1}}$.; $\int_0^a \frac{1}{x^p}\,\mathrm{d}x$ converges iff $p < 1$, and if so, $\int_0^a \frac{1}{x^p}\,\mathrm{d}x = -\frac{1}{(p-1)\,a^{p-1}}$.;

Proof. $\int \frac{1}{x^p}\,\mathrm{d}x = \begin{cases} -\frac{1}{(p-1)\,x^{p-1}}, & p \neq 1 \\ \ln(x), & p = 1 \end{cases}$; $\forall p < 1 : \nexists \lim_{x \to \infty} -\frac{1}{(p-1)\,x^{p-1}}$; $\nexists \lim_{x \to \infty} \ln(x)$; $\forall p > 1 : \lim_{x \to \infty} -\frac{1}{(p-1)\,x^{p-1}} = 0$ $\int_a^\infty \frac{1}{x^p}\,\mathrm{d}x = \frac{1}{(p-1)\,a^{p-1}}$; $\forall p > 1 : \nexists \lim_{x \to 0^+} -\frac{1}{(p-1)\,x^{p-1}}$; $\nexists \lim_{x \to 0^+} \ln(x)$; $\forall p < 1 : \lim_{x \to 0^+} -\frac{1}{(p-1)\,x^{p-1}} = 0$ $\int_0^a \frac{1}{x^p}\,\mathrm{d}x = -\frac{1}{(p-1)\,a^{p-1}}$; □

Generalized p-test (for improper integral)

Define iterated logarithmic function $L_n(x)$ as $L_0(x) = x, \quad x \in \mathbb{R}_{>0}$; $L_n(x) = \ln\left(L_{n-1}(x)\right), \quad n \in \mathbb{N}, x \in \mathbb{R}_{>0}$.; For any fixed $a > 1, k \in \mathbb{N}$, $\int_a^\infty \frac{1}{\left(\prod_{i=0}^{k-1} L_i(x)\right)\left(L_k(x)\right)^m}\,\mathrm{d}x$; converges iff $m > 1$.

Proof. Let $u = L_k(x)$; $\mathrm{d}u = \frac{\mathrm{d}x}{\prod_{i=0}^{k-1} L_i(x)}$; $\int \frac{1}{\left(\prod_{i=0}^{k-1} L_i(x)\right)\left(L_k(x)\right)^m}\,\mathrm{d}x = \int \frac{1}{u^m}\,\mathrm{d}u$; By p-test, the statement holds. □

For any fixed $0 < a < 1$, $\int_0^a \frac{1}{x(\ln x)^m}\,\mathrm{d}x$; converges iff $m < 1$.

Proof. By substitution of $u = \ln x$. □

Generalized p-test for series or generalized p-series test

Define iterated logarithmic function $L_n(x)$ as $L_0(x) = x, \quad x \in \mathbb{R}_{>0}$; $L_n(x) = \ln\left(L_{n-1}(x)\right), \quad n \in \mathbb{N}, x \in \mathbb{R}_{>0}$.; For any fixed $a \in \mathbb{N}$, $\sum_{n=a}^{\infty} \frac{1}{\left(\prod_{i=0}^{k-1} L_i(n)\right)\left(L_k(n)\right)^m}$; converges iff $m > 1$.

Proof. By integral test and generalized p-test for improper integral, the statement holds. □

Generalized Master Theorem

For a positive valued net $T(n)$ defined on $n = b^h$ for any $h \in \mathbb{N}_0$ as $T(n) = aT\left(\frac{n}{b}\right) + f(n)$; for some constants $a > 0$ and $b > 1$, and non-negative-valued function $f(n)$,

If for some $c < \log_b a$, $f(n) = O\left(n^c\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\right)$.;

Define iterated logarithmic function $L_n(x)$ as $L_0(x) = x, \quad x \in \mathbb{R}_{>0}$; $L_n(x) = \ln\left(L_{n-1}(x)\right), \quad n \in \mathbb{N}, x \in \mathbb{R}_{>0}$.;

If for some positive integers k and positive number m, $f(n) = \Theta\left(n^{\log_b a}\left(\prod_{i=1}^{k-1}\left(L_i(n)\right)^{-1}\right)\left(L_k(n)\right)^{m-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\left(L_k(n)\right)^m\right)$.;

If for some positive integer k, $f(n) = \Theta\left(n^{\log_b a}\prod_{i=1}^{k}\left(L_i(n)\right)^{-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\left(\ln n\right)^m\right)$.;

If for some positive integers m, $f(n) = \Theta\left(n^{\log_b a}\left(\ln n\right)^{m-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\left(\ln n\right)^m\right)$.;

If $f(n) = o\left(n^{\log_b a}\left(\ln n\right)^{-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\ln\left(\ln n\right)\right)$.;

If for some positive integers k and some positive number m, $f(n) = \Theta\left(n^{\log_b a}\left(\prod_{i=1}^{k-1}\left(L_i(n)\right)^{-1}\right)\left(L_k(n)\right)^{-m-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\right)$.;

If for some positive integers m, $f(n) = \Theta\left(n^{\log_b a}\left(\ln n\right)^{-m-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\right)$.;

If $f(n) = \Omega\left(n^c\right)$ for some $c > \log_b a$, and the regularity condition that $\exists m \in \mathbb{R}, k < 1$ such that $n \geq m$ implies $af\left(\frac{n}{b}\right) \leq kf(n)$; is satisfied, then $T(n) = \Theta(f(n))$.;

Proof. Let $s = \log_b a$, $n = b^h$, and $f(x) = x^s g(x)$. At level j, there are a^j nodes, each of size $\frac{n}{b^j}$. Thus, the total non-recursive work at level j is $a^j f\left(\frac{n}{b^j}\right)$.; Hence

$$T(n) = \sum_{j=0}^{h-1} a^j f\left(\frac{n}{b^j}\right) + n^s T(1)$$

$$= \sum_{j=0}^{h-1} b^{sj} f\left(\frac{b^h}{b^j}\right) + \Theta\left(n^s\right) \quad ;$$

$$= n^s \sum_{j=0}^{h-1} g\left(b^{h-j}\right) + \Theta\left(n^s\right) = \quad n^s \sum_{r=1}^{h} g\left(b^r\right) + \Theta\left(n^s\right)$$

Case 1: If for some $c < \log_b a$, $f(n) = O\left(n^c\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\right)$.; Case 2: $L_k\left(b^x\right) = \Theta\left(L_{k-1}(x)\right)$; $f(n) = \Theta\left(n^{\log_b a}\left(\prod_{i=1}^{k-1}\left(L_i(n)\right)^{-1}\right)\left(L_k(n)\right)^m\right)$; $g\left(b^r\right) = \Theta\left(\left(\prod_{i=1}^{k-2}\left(L_i(r)\right)^{-1}\right)\left(L_{k-1}(r)\right)^{m-1}\right)$; By generalized p-test for series: Sub-case 1 $m > 0$: $T(n) = \Theta\left(n^{\log_b a}\left(L_k(n)\right)^m\right)$.; Sub-case 2 $m = 0$: $T(n) = \Theta\left(n^{\log_b a}L_{k-1}(n)\right)$.; Sub-case 3 $m = 0$: $T(n) = \Theta\left(n^{\log_b a}\right)$.; Case 3: If $f(n) = \Omega\left(n^c\right)$ for some $c > \log_b a$, and the regularity condition that $\exists m \in \mathbb{R}, 0 < k < 1$ such that $n \geq m$ implies $af\left(\frac{n}{b}\right) \leq kf(n)$; is satisfied, then $af\left(\frac{n}{b}\right) \leq kf(n)$; $a^j f\left(\frac{n}{b^j}\right) \leq a^{j-1}kf\left(\frac{n}{b^{j-1}}\right) \leq k^j f(n)$; $T(n) = \sum_{j=0}^{h-1} a^j f\left(\frac{n}{b^j}\right) + n^s T(1)$ $\quad ; \quad \sum_{j=1}^{\infty} k^j f(n) =$ $= f(n) + \sum_{j=1}^{h-1} O\left(k^j f(n)\right) + \Theta\left(n^s\right)$ $\frac{f(n)}{1-k}$; $T(n) = f(n) + O(f(n)) + \Theta\left(n^s\right) = \Theta(f(n))$; □

Terms:

h is called recursion level.

$\log_b a$ is called critical exponent c_{crit}.

$T(1)$ is called leaf cost.

At each node, the first term is called recursive work, and the second term is called non-recursive work.

For original master theorem, the second case is

If for some positive integers m, $f(n) = \Theta\left(n^{\log_b a}\left(\ln n\right)^{m-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\left(\ln n\right)^m\right)$.;

If $f(n) = o\left(n^{\log_b a}\left(\ln n\right)^{-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\ln\left(\ln n\right)\right)$.;

If for some positive integers m, $f(n) = \Theta\left(n^{\log_b a}\left(\ln n\right)^{-m-1}\right)$.; then $T(n) = \Theta\left(n^{\log_b a}\right)$.;

Stirling's approximation

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n\left(1 + O\left(\frac{1}{n}\right)\right);$$

Python Operator Precedence Order

`()` > `**` > `+x` `-x` `~x` (bitwise NOT) > `*` `/` `//` `%` > `+` `-` > `«` `»` (preserve sign) > `&` (bitwise AND) > `^` (bitwise XOR) > `|` (bitwise OR) > `==` `!=` > `>` `>=` `<` `<=` (value comparison) `is` `is not` (identity) `in` `not in` > `not` > `and` > `or`

```
a = [10, 20, 30]  # list is a
      dynamic array of elements of
      arbitrary types
a[0]  # 10, O(1)
a[1]  # 20, O(1)
a[-1]  # 30, O(1)
a[-2]  # 20, O(1)
len(a) # 3, O(1)
a.__len__() # 3, O(1)
a.append(40)  # a == [10, 20, 30,
      40], amortized O(1)
a.pop()  # a == [10, 20, 30], O(1)
a.insert(1, 50)  # a == [10, 50, 20,
      30], O(n)
a.pop(0)  # a == [50, 20, 30], O(n)
a[0] = 0  # a == [0, 20, 30], O(1)
b, c, d = a  # b == 0, c == 20, d ==
      30
a.clear()  # a == []
a = [0, 1, 2]
b = a  # id(a) == id(b)
b.pop()  # a == b == [0, 1], id(a)
      == id(b)
b = [1, 2]  # id(a) != id(b), a ==
      [0, 1]
a = [1, 1, 1]
```

```
a.count(1)  # 3, O(n)
b = a.copy()  # a == b but id(a) !=
      id(b)
a.index(1)  # 0, O(n), first
      occurrence
a.extend([0, 4, 3, 4]),  # a == [1,
      1, 1, 0, 4, 3, 4], O(k)
a.remove(4)  # a.pop(a.index(x))
a.reverse()  # a == [4, 3, 0, 1, 1,
      1], O(n)
a.sort()  # [0, 1, 1, 1, 3, 4], O(n
      log n) worst-case O(n) best-
      case, Powersort
```

```
sorted(a) # same as a.sort()
del a  # a is now not defined
a = [1, "1", [1]]  # arbitrary types


from array import array
a = array('b', [10, 20, 30]) # array
      .array is a dynamic array of
      elements of the same
      primitive data type in a
      compact, contiguous memory
      buffer
# 'b' char
```

```
# 'B' unsigned char
# 'h' short
# 'H' unsigned short
# 'i' int
# 'I' unsigned int
# 'l' long
# 'f' float
# 'd' double
a.itemsize # 1
a.buffer_info() # (address, len(a))
b = a.tolist() # b == [10, 20, 30]
# As len(a) gets large, array tends
      to be smaller than list
```

```
# Methods for list are available for
      array with same complexity


class Circular:
    def __init__(self, data):
        self.data = data

    def __iter__(self):
        while True:
            yield from self.data
```

```
class ListStack(list):  # LIFO O(1)
    def push(self, x):
        return self.append(x)

    def top(self):
        return self[-1]

    def is_empty(self):
        return self.__len__() == 0
```

```
class _SNode:
    def __init__(self, data, next=
        None):
        self.data = data
        self.next = next


class SinglyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0
```

```
    def __len__(self):
        return self.size

    def is_empty(self):
        return self.head is None

    def prepend(self, x):
        self.head = _SNode(x, self.
            head)
        if self.tail is None:
            self.tail = self.head
        self.size += 1
```

```
    def append(self, x):
        self.tail = _DNode(x, None)
        if self.head is None:
            self.head = self.tail
        self.size += 1

    def pop_front(self):
        if self.head is None:
            raise IndexError("remove
                from empty list"
                )
        x = self.head.data
        self.head = self.head.next
```

```
        self.size -= 1
        return x

    def peek_front(self):
        if self.is_empty():
            raise IndexError("peek
                at empty list")
        return self.head.data

    def peek_back(self):
        if self.is_empty():
            raise IndexError("peek
                at empty list")
```

```
        return self.tail.data

    def clear(self):
        for _ in range(self.size):
            self.pop_front()

    def __iter__(self):
        cur = self.head
        while cur is not None:
            yield cur.data
            cur = cur.next
```

```
class LinkedStack(SinglyLinkedList):
    # LIFO O(1)
    def push(self, x):
        return self.prepend(x)

    def pop(self):
        return self.pop_front()

    def top(self):
        return self.peek_front()


class LinkedQueue(SinglyLinkedList):
```

```
    # FIFO O(1)
    def enqueue(self, x):
        return self.append(x)

    def dequeue(self):
        return self.pop_front()

    def peek(self):
        return self.peek_front()
```

```
class _DNode:
    def __init__(self, data, prev=
        None, next=None):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def __len__(self):
```

```
        return self.size

    def is_empty(self):
        return self.size == 0

    def prepend(self, x):
        self.head = _DNode(x, None,
            self.head)
        if self.tail is None:
            self.tail = self.head
        self.head.next.prev = self.
            head
        self.size += 1

    def append(self, x):
```

```
        self.tail = _DNode(x, self.
            tail, None)
        if self.head is None:
            self.head = self.tail
        self.size += 1

    def pop_front(self):
        if self.head is None:
            raise IndexError("pop
                from empty list")
        x = self.head.data
        self.head = self.head.next
        self.size -= 1
        if self.head is None:
            self.tail = None
```

```
        else:
            self.head.prev = None
        return x

    def pop_back(self):
        if self.tail is None:
            raise IndexError("pop
                from empty list")
        x = self.tail.data
        self.tail = self.tail.prev
        self.size -= 1
        if self.tail is None:
            self.head = None
        else:
            self.tail.next = None
```

```
        return x

    def peek_front(self):
        if self.is_empty():
            raise IndexError("peek
                at empty list")
        return self.head.data

    def peek_back(self):
        if self.is_empty():
            raise IndexError("peek
                at empty list")
        return self.tail.data

    def clear(self):
```

```
        for _ in range(self.size):
            self.pop_back()

    def __iter__(self):
        cur = self.head
        while cur is not None:
            yield cur.data
            cur = cur.next


class LinkedDequeue(DoublyLinkedList
    ):  # Both sides IO O(1)
    def push(self, x):
        return self.prepend(x)
```

```
    def enqueue(self, x):
        return self.append(x)

    def dequeue(self):
        return self.pop_front()

    def pop(self):
        return self.pop_front()

    def top(self):
        return self.peek_front()

    def peek(self):
        return self.peek_front()
```

```
class ListDequeue:  # Both sides IO
    amortized O(1)
    def __init__(self, block_size
        =64, front_step=1,
        back_step=1):
        assert block_size > 0
        assert front_step > 0
        assert back_step > 0
        self.block_size = block_size
        self.front_step = front_step
        self.back_step = back_step
        self.data = [
            [None] * self.block_size
            for _ in range(self.
                front_step + self
                .back_step - 1)
        ]
        self.block_begin = 0
        self.block_end = self.
            block_begin + 1
        self.in_begin = (self.
            block_size - 1) // 2
        self.in_end = self.in_begin
        self.size = 0

    def __len__(self):
        return self.size

    def is_empty(self):
```

```
        return self.size == 0

    def capacity(self):
        return self.block_size * len
            (self.data)

    def is_valid(self):
        return (
            self.size
            == self.block_size * (
                self.block_end -
                self.block_begin
                - 1)
            + self.in_end
            - self.in_begin
        )

    def reserve_front(self, blocks=
        None):
        if blocks is None:
            blocks = self.front_step
        new = [[None] * self.
            block_size for _ in
            range(blocks)]
        new.extend(self.data)
        self.data = new
        self.block_begin += blocks
        self.block_end += blocks
```

```
    def reserve_back(self, blocks=
        None):
        if blocks is None:
            blocks = self.back_step
        self.data.extend([[None] *
            self.block_size for _
            in range(blocks)])

    def shrink_front_to_fit(self):
        new = self.data[self.
            block_begin :]
        self.data = new
        self.block_end -= self.
            block_begin
        self.block_begin = 0

    def shrink_back_to_fit(self):
        for i in range(self.
            block_end, len(self.
            data)):
            self.data.pop()

    def prepend(self, x):
        if self.in_begin != 0:
            self.in_begin -= 1
        else:
            if self.block_begin ==
                0:
                self.reserve_front()
```

```
            self.block_begin -= 1
            self.in_begin = self.
                block_size - 1
        self.data[self.block_begin][
            self.in_begin] = x
        self.size += 1

    def append(self, x):
        if self.in_end != self.
            block_size:
            self.in_end += 1
        else:
            if self.block_end == len
                (self.data):
                self.reserve_back()
            self.block_end += 1
            self.in_end = 1
        self.data[self.block_end -
            1][self.in_end - 1] =
            x
        self.size += 1

    def pop_front(self):
        if self.is_empty():
            raise IndexError("pop
                from empty deque"
                )
        x = self.data[self.
            block_begin][self.
```

```
            in_begin]
        self.data[self.block_begin][
            self.in_begin] = None
        self.size -= 1
        if self.in_begin != self.
            block_size - 1:
            self.in_begin += 1
        else:
            self.block_begin += 1
            self.in_begin = 0
        return x

    def pop_back(self):
        if self.is_empty():
            raise IndexError("pop
                from empty deque"
                )
        x = self.data[self.block_end
            - 1][self.in_end -
            1]
        self.data[self.block_end -
            1][self.in_end - 1] =
            None
        self.size -= 1
        if self.in_end != 1:
            self.in_end -= 1
        else:
            self.block_end -= 1
            self.in_end = self.
```

```
                block_size
        return x

    def peek_front(self):
        if self.is_empty():
            raise IndexError("peek
                from empty deque"
                )
        return self.data[self.
            block_begin][self.
            in_begin]

    def peek_back(self):
        if self.is_empty():
            raise IndexError("peek
                from empty deque"
                )
        return self.data[self.
            block_end - 1][self.
            in_end - 1]

    def clear(self):
        for _ in range(self.size):
            self.pop_back()

    def __iter__(self):
        block = self.block_begin
        index = self.in_begin
        for _ in range(self.size):
```

```
            yield self.data[block][
                index]
            index += 1
            if index == self.
                block_size:
                index = 0
                block += 1

    def push(self, x):
        return self.prepend(x)

    def enqueue(self, x):
        return self.append(x)

    def dequeue(self):
        return self.pop_front()

    def pop(self):
        return self.pop_front()

    def top(self):
        return self.peek_front()

    def peek(self):
        return self.peek_front()
```